SPI-CODE · VISUAL DESIGN · EXPLORATORY PROPOSAL

SPI Code visual design

The window, the Shore palette in semantic roles, and the component kit, with the T3 Code or Keelson source each piece is built from.

Product brief · Product requirements · **Visual design** (this page) · Original combined design (PDF)

The program is read today through the keelson-rib-spi console, which draws boards inside Keelson's own window, and through a terminal. The original combined design sketched an SPI Code window in two mockups. Its inspector gave the wrong fix path for a Prove failure, it had no dark or code-thread view, and several of its light-mockup text colors fell below WCAG AA contrast.

This document sets the window's layout, the colors and their jobs, and the components the requirements name. The structure comes from T3 Code: its three-pane layout, right-panel tabs, approval strip, work log and terminal drawer, and its semantic theme roles. The colors come from Keelson's Shore palette, with darker text variants where Shore's light values are fills. Each piece names the file it is copied from, ported from or modeled on, so the window can be assembled from working code.

Every mockup on this page is a drawn proposal, not a screenshot; names, counts, SHAs and PR numbers are examples. It does not specify icons glyph by glyph, motion timing or the onboarding copy. The contrast figures are computed from the hex values with the WCAG 2 formula; no built window has been measured.

COLORS IN THE MOCKUPS

- P0, error, a failed read
- P1, waiting, a pending card, stale evidence
- P2, selection, primary action, anchor
- Healthy, applied, fresh evidence
- Copilot's own words
- Inferred, not read

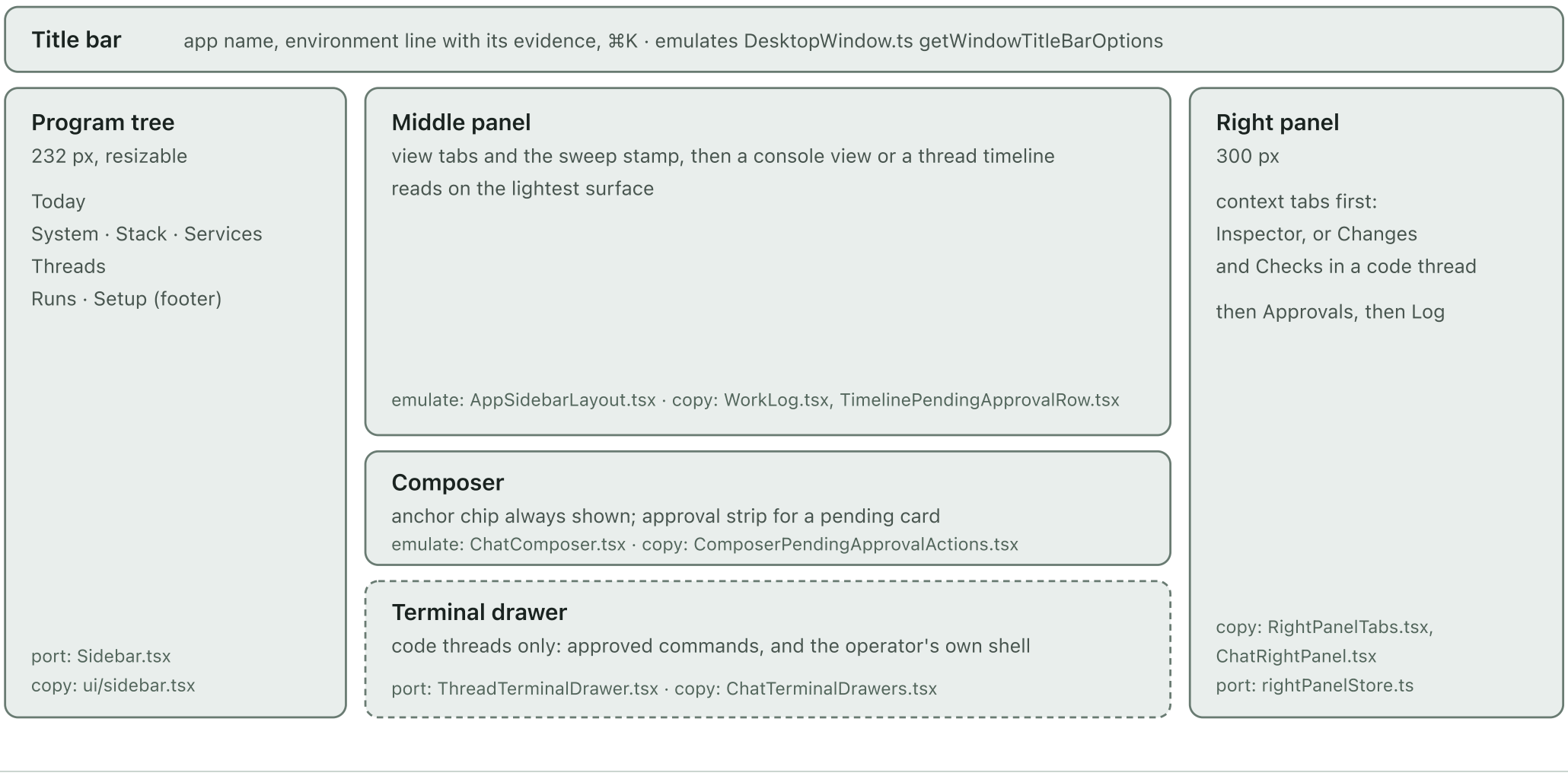
Where the window comes from

Three sources supply nearly everything. The verbs match the source and borrow map in the product requirements: **Copy** takes the file and keeps its MIT notice, **Port** rewrites it without Effect or T3 Code's provider model, **Emulate** builds the idea fresh, and **Reference** is read only.

Source	What it supplies	How SPI Code uses it
T3 Code `/Users/danielscholl/source/github/t3code` at `43bf0babd4`	Window layout, sidebar, right-panel tabs, approval strip and timeline row, work log, terminal drawer, changed-files tree, command palette, theme role model, title bar options	**Copy** components that import no Effect; **Port** the sidebar, terminal drawer, command palette and right-panel store; **Emulate** the role model, cut down. Appendix A lists each file.
Keelson `/Users/danielscholl/keelson/keelson` at `da5da474`	Shore token values for both appearances, WCAG contrast and palette validation functions	**Copy** the Shore values and `palette.ts` into the app's theme package; the validator runs in CI.
Original combined design `design/spi-code-design.html` at `119b304`; the PDF beside this page is a print of it	The window mockup, Setup mockup and their CSS	The mockups below reuse its markup and CSS with corrected text and contrast.

Window anatomy

The window has five regions. A console view or a thread fills the middle; the left tree chooses what the middle shows, and the right panel describes the selection in the middle. The terminal drawer exists only in code threads.



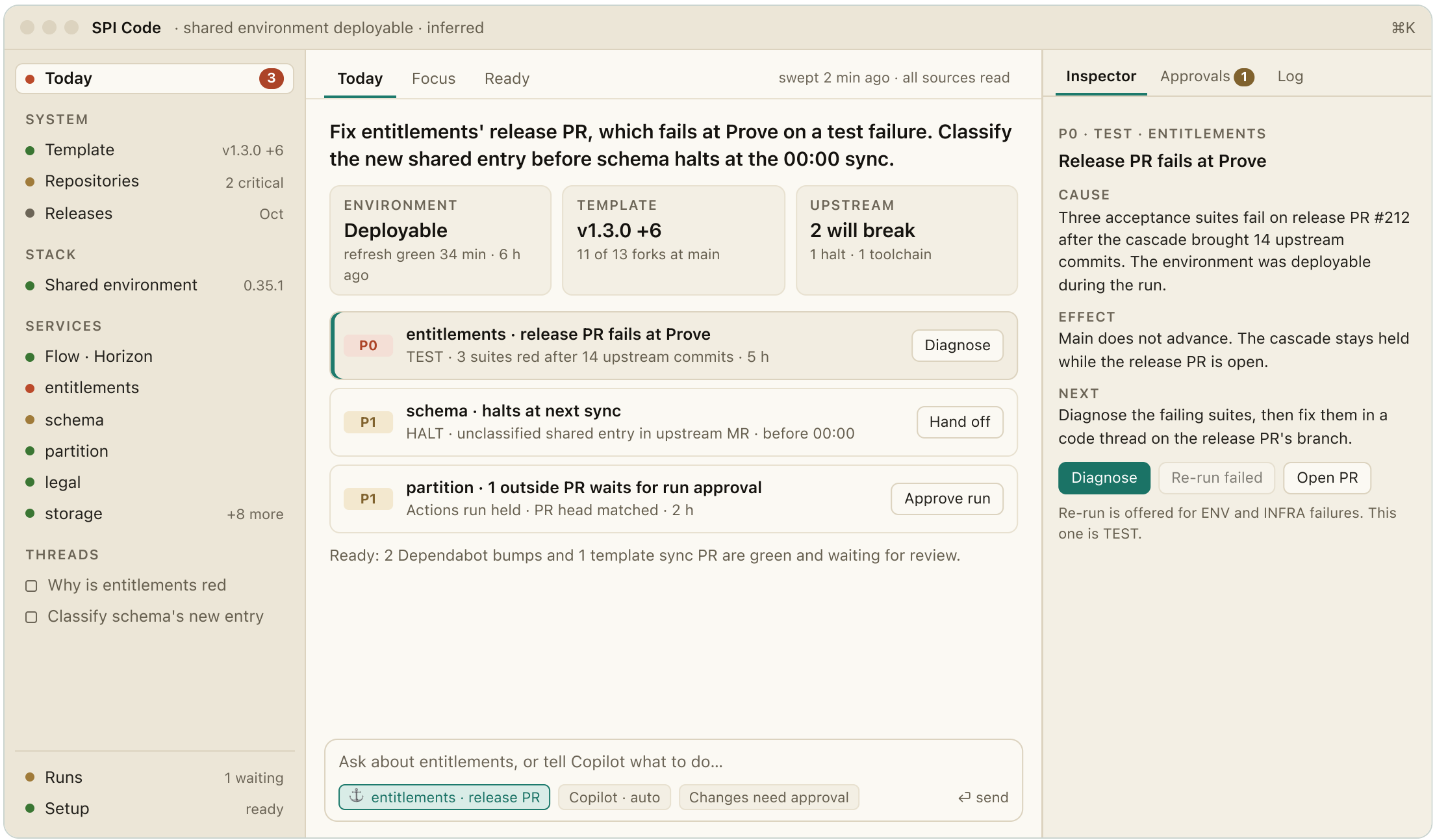
These rules hold in every view:

- Reading happens in the middle panel on the card surface, the lightest one in the light appearance. The tree, title bar and right panel use the chrome surfaces around it.
- One selection drives the right panel. Selecting a queue row, a fork or a run fills the inspector; nothing opens a modal.
- Every view states the sweep time and whether every source was read, in the view tab row.
- The composer always shows its anchor, so the operator knows what context Copilot starts from.
- A disabled action states why, in place.
- Color never carries meaning alone. Priorities are written as P0, P1 and P2; card states and evidence chips are words.

On macOS the title bar uses `hiddenInset` with the traffic lights inset, and on Windows a hidden title bar with `titleBarOverlay`, as T3 Code's `apps/desktop/src/window/DesktopWindow.ts` `getWindowTitleBarOptions` (line 235) does. Its heights there are 52 and 40 pixels; SPI Code's are not set yet. Below a window width of 980 pixels the right panel becomes a popover, using T3 Code's `RIGHT_PANEL_INLINE_LAYOUT_MEDIA_QUERY` from `apps/web/src/rightPanelLayout.ts`, and the tree can collapse to T3 Code's 3rem icon rail (`ui/sidebar.tsx` `SIDEBAR_WIDTH_ICON`, line 30).

Today

Today is the default view. It shows the briefing headline, three fact tiles, the focus queue in P0, P1, P2 order, and the ready line. The selected row fills the inspector. This mockup corrects the original's inspector, which sent a Prove failure to `fork_integration`; a Prove failure on the open release PR is fixed on that PR's head branch (product requirements, Code threads).



Proposed, light appearance. The tree's status dots come from the latest sweep: red has a P0, amber a P1, green nothing waiting, gray nothing measured. The Today pip counts focus items; the Approvals pip counts pending cards and shows the same number on every screen.

Program tree

The tree follows the program's nouns, not repositories or chat sessions. Each entry renders from the same core model the rib console uses.

Group	Entries	Count or label on the right	Model in @spi-code/core
Today	Briefing, focus queue, ready line	Focus items, as a red pip	`buildBriefing`, `model/queue.ts`
System	Template, Repositories, Releases	Template version and drift, critical alerts, release month	`model/rollout.ts`, security, release notes
Stack	Shared environment	Running stack version	`model/stack.ts`
Services	Flow and Horizon, then one entry per service line	Overflow count after the first five	`model/lines.ts`, `model/radar.ts`
Threads	Operate and code threads, newest first	None; a square marker, not a dot	ThreadStore
Footer	Runs, Setup	Runs waiting; Setup readiness	IntentLedger; Setup probes

The row shape, the selected state and the collapse behavior come from T3 Code's `Sidebar.tsx` and `ThreadStatusIndicators.tsx`. Both import Effect and are built around projects and chat threads, so they are ported. The `ui/sidebar.tsx` primitive is copied.

Right panel

The right panel's tabs come in a fixed order: the context tabs for what is selected, then Approvals, then Log. On a console view the context tab is Inspector. In a code thread it is Changes and Checks.

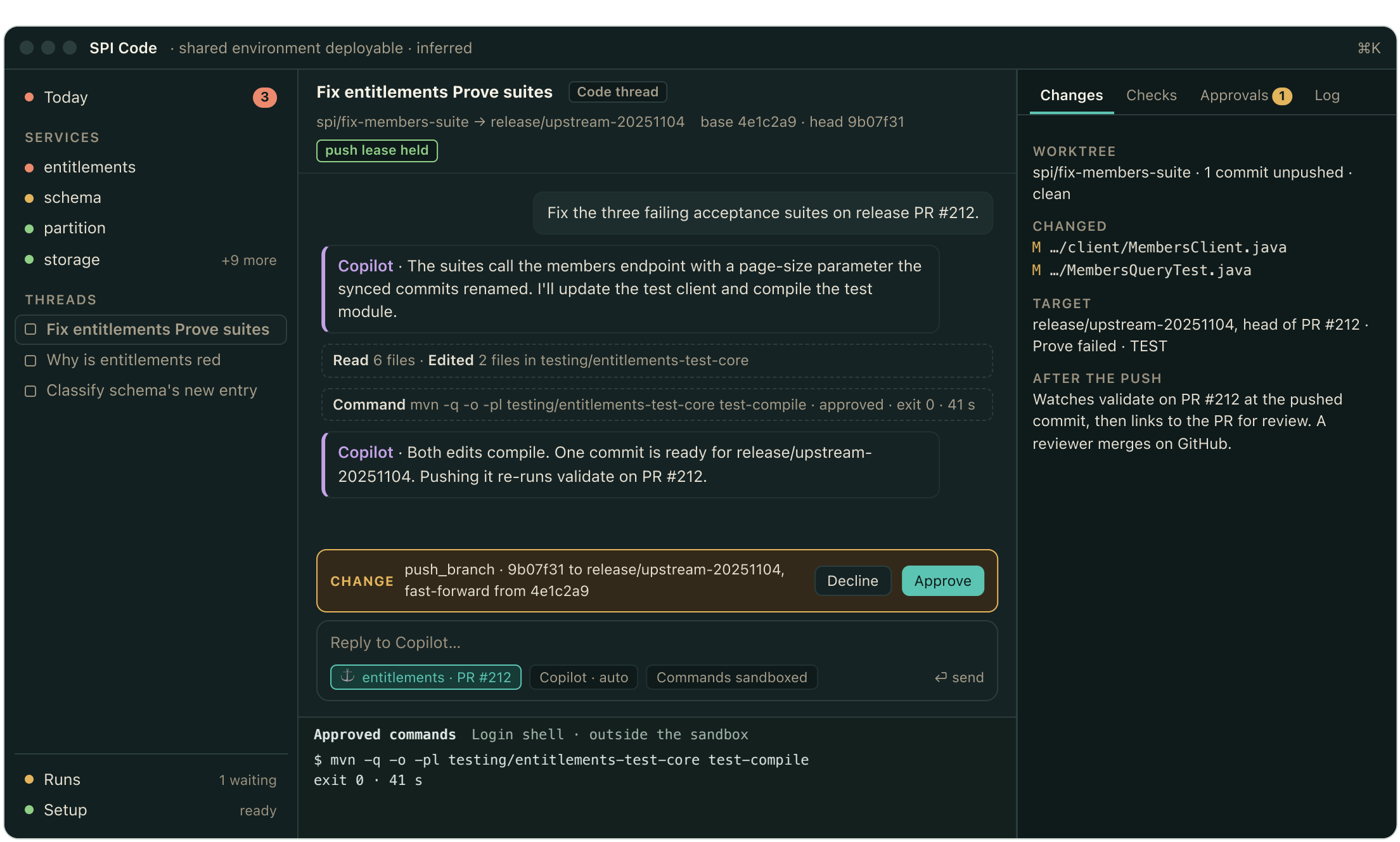
Approvals is one inbox for every pending card in the app, and Log is the selected object's runs and the sweep's read history for it.

A card has one id and one state, held in the host's ApprovalStore, and appears wherever it is relevant. It is decided in one place per origin. A card requested in a thread is decided in that thread's composer strip, and its timeline row and inbox row point there. A card requested by a button is decided on the card itself. A button never writes; it creates the card. The card states are drawn in the product requirements, Authority, tiers and approvals.

T3 Code's `RightPanelTabs.tsx` and `ChatRightPanel.tsx` import no Effect and are copied. `rightPanelStore.ts` (1,258 lines) is ported down to the selection and each context's chosen tab, which is all the window keeps itself.

Code thread

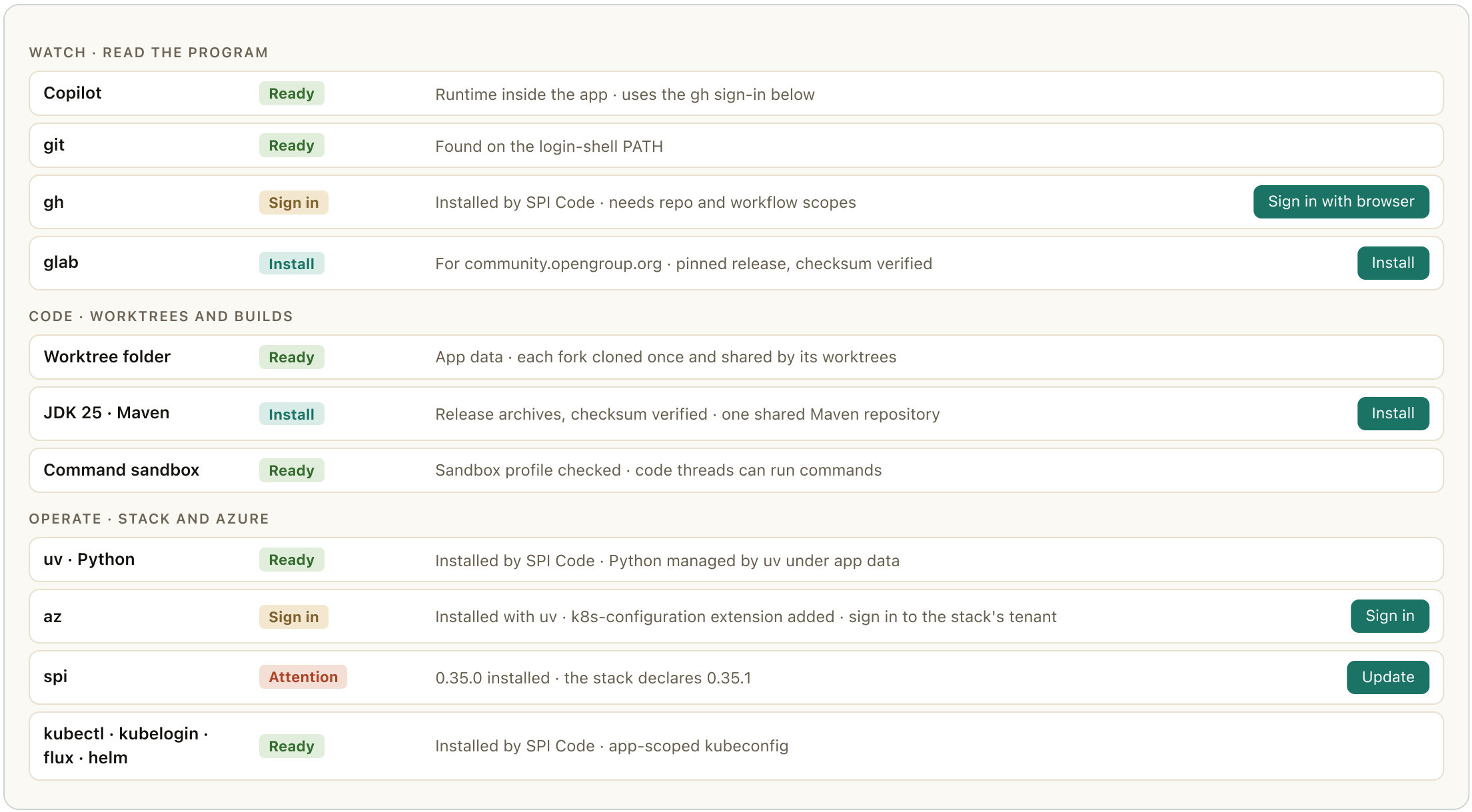
A code thread works in a worktree on one fork branch. Its header names the local branch, the remote target, and the base and head. The timeline interleaves the operator's messages, Copilot's replies, work log entries and cards. A pending card sits in the composer strip. The drawer has two tabs: the read-only record of approved commands, and the operator's own login shell, labeled as outside the sandbox.



Proposed, dark appearance. The push card is a Change-tier card decided in the composer strip; it names the exact commit and the commit the remote must still be at. The terminal follows the appearance, as T3 Code's `terminalThemeFromApp` in `ThreadTerminalDrawer.tsx` does.

Setup

Setup lists every tool the app depends on, grouped by what it enables, with one state and one action per row. A group that is not ready disables the views that need it and says so.



Proposed, light appearance. Install, Sign in and Attention are the only states that carry an action. What each install does is in the product requirements, Setup and tools.

The readiness states follow T3 Code's provider installation model (`apps/server/src/provider/CodexInstallation.ts`, ported). The first-run flow is modeled on `apps/web/src/components/onboarding/FirstRunGate.tsx` and `WelcomeWizard.tsx`, and a status banner on `chat/ProviderStatusBanner.tsx` reports a Copilot sign-in that lapses after setup.

Shore in semantic roles

SPI Code has one palette, Shore, in a light and a dark appearance. The appearance follows the system unless Settings pins one; there is no palette picker. Components never name a hex value or a Shore key. They name a role, following T3 Code's role model in `packages/shared/src/themePalettes.ts` (`THEME_COLOR_ROLES`, line 30), cut down to the roles this app uses. T3 Code has error, warning and update roles but no success role, so SPI Code adds one. It also adds an agent role for Copilot's words.

Role	Used for	Shore key	Light	Dark	Text variant
`chrome`	Title bar, right panel, tile backgrounds	`bg`	`#f3efe4`	`#0f1a1b`	
`sidebar`	Program tree, title bar strip	`bgSoft`	`#ece6d7`	`#132021`	
`surface`	Middle panel, cards, rows	`card`	`#fbf9f3`	`#162324`	
`surfaceRaised`	Selected row, own messages	`card2`	`#f2ede1`	`#1d2d2e`	
`border`	Panel edges, card outlines	`border`, `borderSoft`	`#dcd3bf`	`#2b3d3d`	
`text`	Body text, titles	`fg`, `fgStrong`	`#23211c`	`#e4dfd1`	Same
`textMuted`	Secondary lines, reasons	`muted`	`#665f4f`	`#a59d8a`	Same
`placeholder`	Group labels, stamps, counts	`dim`	`#8a8170`	`#7d7666`	`#6c6557` · `#9a927f`
`accent`	Selection, primary action, anchor, P2	`accent`	`#1d7a6c`	`#5cc4b2`	`#1b7366` · same
`error`	P0, binding failed, failed read	`red`	`#bb4a2b`	`#ec8a6e`	`#ac4428` · same

Role	Used for	Shore key	Light	Dark	Text variant
`warning`	P1, pending card, stale evidence	`yellow`	`#a07b3a`	`#e2b45c`	`#80622e` · same
`success` (new)	Healthy, applied, fresh evidence	`green`	`#3a7633`	`#8fcf86`	`#356c2f` · same
`agent` (new)	Copilot's messages and proposals	`magenta`	`#8a5cc0`	`#c3a2e8`	`#7e4cba` · same

Shore's light accent, red, yellow, green, magenta and dim values work as fills, dots and borders, but all six fall below 4.5:1 as small text on at least one light surface. Each such role gets a darker text variant in the light appearance; dark Shore values pass as text and are used unchanged, except `dim`, which is lightened. Appendix B has the computed ratios. A filled badge or primary button puts the light-appearance text variant behind white text, and the dark value behind the dark chrome color.

chrome
`#f3efe4` · `#0f1a1b`

surface
`#fbf9f3` · `#162324`

text
`#23211c` · `#e4dfd1`

accent
`#1d7a6c` · `#5cc4b2`

error · P0
`#bb4a2b` · `#ec8a6e`

warning · P1
`#a07b3a` · `#e2b45c`

success
`#3a7633` · `#8fcf86`

agent · Copilot
`#8a5cc0` · `#c3a2e8`

Each swatch shows light, then dark. Values from Keelson `packages/shared/src/design-tokens.ts`, `SHORE_LIGHT` (line 154) and `SHORE_DARK` (line 124).

Charts in the Stack and Releases views draw from Shore's six-color `series`. CI checks every text pair in the table with Keelson's `wcagContrast` and the series with `validateCategoricalPalette`, both from `packages/shared/src/palette.ts` (lines 84 and 189), so a token change that breaks contrast or color-blind separation fails the build (product requirements, NF-07).

Component kit

The same components in both appearances. Each is a role-based React component; none carries its own colors.

QUEUE ROWS

P0 **entitlements · release PR fails at Prove**
TEST · 3 suites red · 5 h — Diagnose

P2 **legal · template sync PR is green**
Ready for review · 1 d — Open PR

EVIDENCE CHIPS

fresh · 2 min | stale · 3 h | incomplete · 40 of 52 | read failed · Actions | unread · GitLab | inferred

TIER BADGES

Read | Local | Command | Change | Consequential | Not offered

APPROVAL CARDS

Re-run cascade PENDING
partition · fork_integration at 7c21d0e
Decline | Approve

Push branch EXECUTING
9b07f31 to release/upstream-20251104

Approve run BINDING FAILED
PR head moved from 3f9a1b2 to 8d04c6e

Classify entry APPLIED
schema · upstream-filter.yml · PR #87

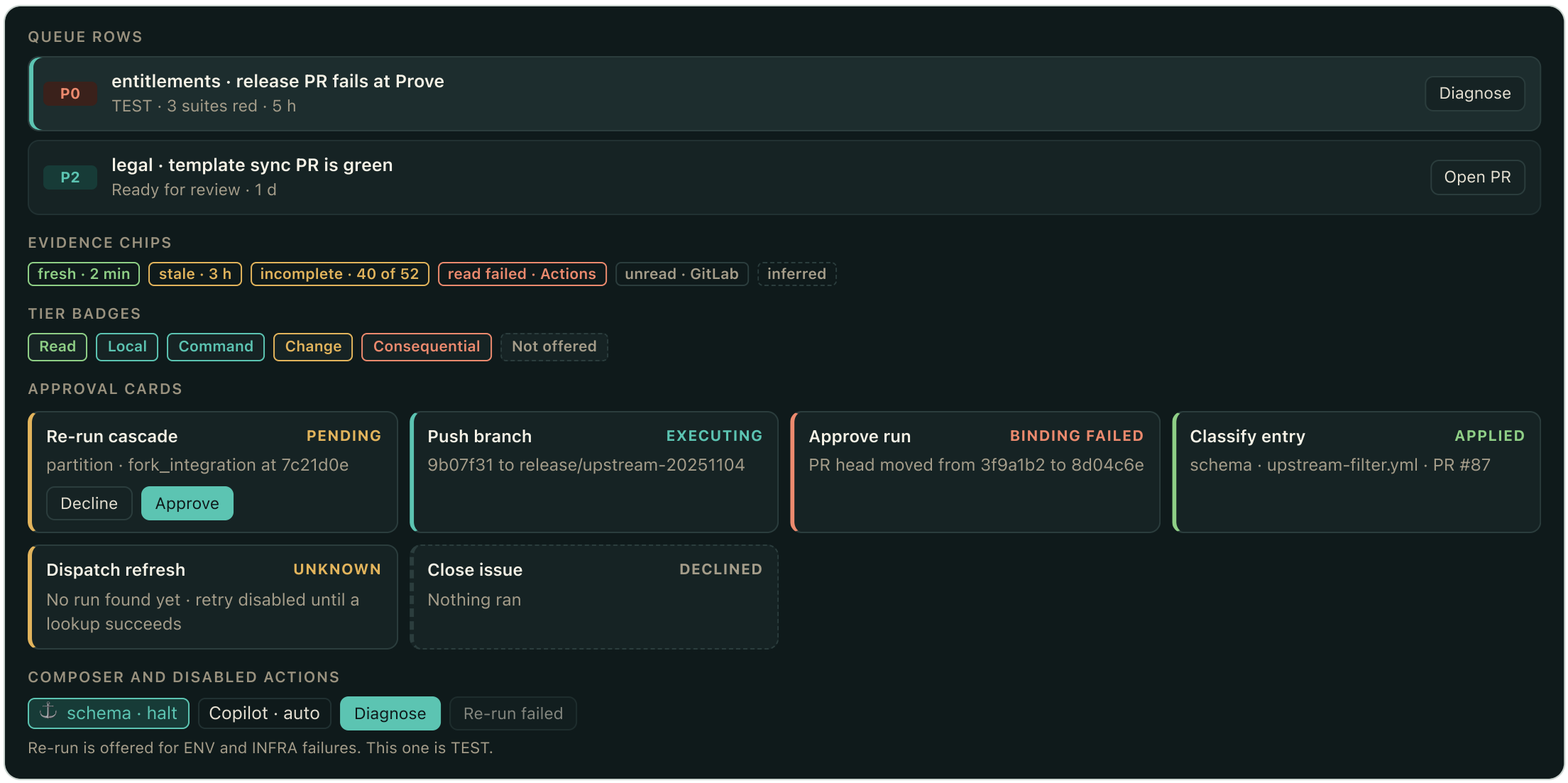
Dispatch refresh UNKNOWN
No run found yet · retry disabled until a lookup succeeds

Close issue DECLINED
Nothing ran

COMPOSER AND DISABLED ACTIONS

schema · halt | Copilot · auto | Diagnose | Re-run failed

Re-run is offered for ENV and INFRA failures. This one is TEST.



Proposed. Card titles, branches and SHAs are examples. Not applied and Invalidated look like Declined with their own word.

Component	Behavior	Built from
Queue row	Priority written as P0 to P2, title, class and cause with age, one primary action. The selected row gets the accent bar and fills the inspector.	Emulates the rib console's focus rows; data from `model/queue.ts`
Fact tile	Label, value, one line of evidence	Emulates the rib briefing's facts
Inspector	Class, title, cause, effect, next action, actions, and a reason under any disabled action	Copies `chat/ChatRightPanel.tsx` for the frame; content from `readItem`
Evidence chip	Fresh, stale, incomplete, failed, unread or inferred, with the age or count. Inferred is a dashed outline, not a color.	Emulated; the dot form follows `ConnectionStatusDot.tsx`
Tier badge	One of six tiers, outlined, on every card and tool row	Emulated; tiers from the product requirements, appendix A
Approval card	Title, state word, bindings in one line, actions only while pending. The left edge carries the state color.	Copies `chat/TimelinePendingApprovalRow.tsx` and `chat/ComposerPendingApprovalActions.tsx`, extended from approve and decline to SPI Code's eight states
Composer	Text field, anchor chip, model chip, mode chip, approval strip above it when a card is pending	Emulates `chat/ChatComposer.tsx` (3,587 lines; SPI Code needs no attachments, slash menu or provider picker)
Work log entry	Dashed row per batch of reads, edits or an approved command, with its result	Copies `chat/WorkLog.tsx` (170 lines)
Changes list	Changed files with status letters; opens a diff	Copies `chat/ChangedFilesTree.tsx`; ports `DiffPanel.tsx`
Terminal drawer	Two tabs, approved commands and login shell, the second labeled outside the sandbox	Ports `ThreadTerminalDrawer.tsx`; copies `chat/ChatTerminalDrawers.tsx` and `chat/ShellCommandBlock.tsx`
Command palette	⌘K or Ctrl+K: go to any tree entry, thread or tool	Ports `CommandPalette.tsx` (2,901 lines, imports Effect)
Setup row	Tool, state, reason, at most one action	Emulates the original design's Setup mockup; readiness from `CodexInstallation.ts`

Type and spacing

SPI Code uses T3 Code's type scale and geometry tokens from `apps/web/src/index.css`, so copied components keep their proportions.

Token	T3 Code value	SPI Code use
`--font-sans`	`-apple-system, BlinkMacSystemFont, "Inter Variable", "Segoe UI", system-ui` (line 171)	All UI text. Whether to bundle Inter is open.
`--font-mono`	`ui-monospace, "SF Mono", Menlo, Consolas` (line 173)	Branches, SHAs, commands, file paths, the terminal
`text-sm`, `text-xs`	14 px, 12 px (Tailwind)	Reading text in threads and the inspector; rows and tabs
`--text-2xs`, `--text-3xs`	11 px, 10 px (lines 183 to 186)	Metadata, counts and chips; group labels and badges
`--radius`, `--control-radius`	0.5rem (lines 1115 and 103)	Cards, rows, buttons, chips
`--sidebar-content-inset`	0.5rem (line 104)	Tree row inset and gap
`SIDEBAR_WIDTH`	16rem; icon rail 3rem (`ui/sidebar.tsx` line 28)	Tree at 232 px by default so all three panes fit at 1280 px; the rail as in T3 Code
`--ease-drawer`	`cubic-bezier(0.32, 0.72, 0, 1)`	The terminal drawer and popovers. Reduced-motion turns it off.

Icons come from `lucide-react`, which T3 Code already uses (`apps/web/package.json`), so copied components need no icon changes.

Open questions

Question	Why it matters	Who decides, by when
Bundle Inter Variable, or use the system stack only?	T3 Code bundles `@fontsource-variable/inter`. Without it, Windows renders Segoe UI and macOS SF, and copied components' widths shift slightly.	Maintainer, before M2
Which terminal renderer does the drawer use?	T3 Code builds libghostty to WebAssembly (`build:ghostty-wasm` in `apps/web/package.json`). Porting that adds a native build step; xterm.js is the common alternative.	Maintainer, before M5
Title bar heights	T3 Code uses 52 px on macOS and 40 px on Windows. The mockups use 34 px; a lower bar leaves more room for the tree but crowds the traffic lights.	Maintainer, at the M0 spike
Does the agent role need its own name, or reuse T3 Code's `accent` slots?	A separate role keeps Copilot's words visually apart from SPI Code's own selection color, which the requirements rely on to tell inference from read facts.	Maintainer, before M3

Reference detail

Appendix A: window reference map

Paths are relative to each repository's root. T3 Code at `43bf0babd4`, Keelson at `da5da474`. Line counts were read with `wc -l` at those commits. "Imports Effect" was checked with a grep for `from "effect` in each file.

Piece	Source	Action	Notes
Title bar options	T3 `apps/desktop/src/window/DesktopWindow.ts` `getWindowTitleBarOptions` (line 235)	Emulate	hiddenInset on macOS, overlay on Windows
App layout	T3 `apps/web/src/components/AppSidebarLayout.tsx` (482)	Emulate	Imports Effect
Program tree	T3 `components/Sidebar.tsx` (3,648), `ThreadStatusIndicators.tsx` (893)	Port	Both import Effect

Piece	Source	Action	Notes
Sidebar primitive	T3 `components/ui/sidebar.tsx` (867)	**Copy**	One Effect import to remove
Right-panel tabs	T3 `components/RightPanelTabs.tsx` (1,577), `chat/ChatRightPanel.tsx` (51)	**Copy**	No Effect; cut to SPI Code's tabs
Right-panel state	T3 `apps/web/src/rightPanelStore.ts` (1,258), `rightPanelLayout.ts` (3)	**Port**	Keep the 980 px breakpoint
Approval strip and timeline row	T3 `chat/ComposerPendingApprovalActions.tsx` (141), `chat/TimelinePendingApprovalRow.tsx` (122)	**Copy**	Extend to eight card states
Composer	T3 `chat/ChatComposer.tsx` (3,587)	**Emulate**	Take the layout, not the code
Work log	T3 `chat/WorkLog.tsx` (170)	**Copy**	
Changed files and diff	T3 `chat/ChangedFilesTree.tsx` (419), `DiffPanel.tsx` (1,495)	**Copy** / **Port**	DiffPanel imports Effect
Terminal drawer	T3 `components/ThreadTerminalDrawer.tsx` (1,914) `terminalThemeFromApp` (line 320); `chat/ChatTerminalDrawers.tsx` (107); `chat/ShellCommandBlock.tsx` (62)	**Port** / **Copy**	Renderer is open; PTY from `apps/server/src/terminal/NodePtyAdapter.ts`
Command palette	T3 `components/CommandPalette.tsx` (2,901)	**Port**	Imports Effect
Status dot	T3 `components/ConnectionStatusDot.tsx` (75)	**Reference**	
Provider status banner	T3 `chat/ProviderStatusBanner.tsx` (192)	**Copy**	For a lapsed Copilot sign-in
First run	T3 `components/onboarding/FirstRunGate.tsx`, `WelcomeWizard.tsx`	**Emulate**	Leads into Setup
Appearance setting	T3 `components/settings/AppearanceSettingsPanel.tsx` (486)	**Port**	Cut to System, Light, Dark
Theme role model	T3 `packages/shared/src/themePalettes.ts` `THEME_COLOR_ROLES` (line 30)	**Emulate**	Cut down; adds success and agent
Type and geometry tokens	T3 `apps/web/src/index.css` (lines 103, 171 to 186, 1115)	**Copy**	
Shore values	Keelson `packages/shared/src/design-tokens.ts` `SHORE_DARK` (124), `SHORE_LIGHT` (154), `DESIGN_TOKEN_CSS_VARS` (198)	**Copy**	Plus the light text variants in appendix B
Shore as CSS	Keelson `apps/web/src/app.css` `[data-palette="shore"]` (lines 157 and 215)	**Reference**	
Contrast and palette checks	Keelson `packages/shared/src/palette.ts` `wcagContrast` (84), `validateCategoricalPalette` (189)	**Copy**	Runs in CI (NF-07)
Mockup markup and CSS	`design/spi-code-design.html` at `119b304`, the app mockup and Setup styles	**Reference**	This page's mockups

Appendix B: text contrast

Computed with the WCAG 2 relative luminance formula. Each ratio is the lowest across the four light surfaces (`#f3efe4`, `#ece6d7`, `#fbf9f3`, `#f2ede1`) or the four dark surfaces (`#0f1a1b`, `#132021`, `#162324`, `#1d2d2e`). AA for small text is 4.5:1.

Role	Shore value as text, light	Text variant, light	Dark value as text
`placeholder`	`#8a8170` · 3.09	`#6c6557` · 4.64	`#7d7666` · 3.17; lightened to `#9a927f` · 4.63
`accent`	`#1d7a6c` · 4.16	`#1b7366` · 4.57	`#5cc4b2` · 6.81
`warning`	`#a07b3a` · 3.13	`#80622e` · 4.55	`#e2b45c` · 7.44
`error`	`#bb4a2b` · 4.09	`#ac4428` · 4.67	`#ec8a6e` · 5.74
`success`	`#3a7633` · 4.42	`#356c2f` · 5.05	`#8fcf86` · 7.80
`agent`	`#8a5cc0` · 3.86	`#7e4cba` · 4.65	`#c3a2e8` · 6.58

Role	Shore value as text, light	Text variant, light	Dark value as text
`textMuted`	`#665f4f` · 5.09	Same	`#a59d8a` · 5.31
`text`	`#23211c` · 12.92	Same	`#e4dfd1` · 10.75

Filled pairs: white on `#1b7366` is 5.69, on `#ac4428` 5.81, on `#80622e` 5.67. Dark chrome `#0f1a1b` on `#e2b45c` is 9.22 and on `#ec8a6e` 7.12. Tinted badges in the light appearance: `#ac4428` on `#f4dfd6` 4.53, `#80622e` on `#f3e8cf` 4.66, `#1b7366` on `#d9ece7` 4.63, `#356c2f` on `#e1eedc` 5.23. The original combined design used the Shore values directly as text and white on `#a07b3a` (3.9) for the Approvals count.